

FieldProof

Autonomous Job Closeout Agent for Field-Service Teams

Version: 1.0

Product Type: Agentic workflow platform

Hackathon: Agents for Humans

Primary Track: Professional Agents

Status: Hackathon MVP

Core Promise: FieldProof completes the administrative work after a field technician completes the physical work, escalating only decisions that genuinely require a human.

1. Executive Summary

Field technicians regularly finish the physical part of a job while leaving significant administrative work behind.

A completed installation, repair, inspection, or maintenance visit can generate:

- site photos
- voice notes
- receipts
- parts lists
- checklists
- customer signatures
- work-order notes
- invoices
- compliance documents

Someone must then determine whether the work is actually complete, whether the evidence matches the job requirements, whether costs are within authorization, whether anything is missing, and whether the work order can be safely closed.

This process is repetitive, slow, and error-prone.

FieldProof is an autonomous closeout agent that performs this work end-to-end.

After a technician marks a job as complete, FieldProof:

1. retrieves the original work order,
2. collects all submitted evidence,
3. understands photos, documents, receipts, and voice notes,
4. converts the evidence into structured claims,

5. compares those claims against job requirements and policies,
6. requests missing evidence automatically,
7. resolves safe discrepancies when possible,
8. escalates only material conflicts to a human,
9. executes approved actions,
10. closes the work order,
11. generates the final closeout package,
12. creates an auditable evidence receipt explaining why every action happened.

FieldProof's core principle is:

A job cannot be closed unless every required claim is supported by compatible evidence or a human explicitly resolves the conflict.

The product is not primarily a chatbot.

It is a background agent that completes work.

2. Problem

A field-service organization may send technicians to:

- install solar equipment,
- repair HVAC systems,
- maintain telecom infrastructure,
- inspect properties,
- repair appliances,
- service industrial machinery,
- install electrical equipment,
- perform construction work,
- complete utility maintenance.

The technician completes the physical work, but operational staff still need to verify:

- Was every required task completed?
- Are the required photos present?
- Do photos actually support the completion claim?
- Were the authorized parts used?
- Do receipts match reported costs?
- Was additional work performed?
- Was additional spending authorized?
- Did the customer sign?
- Are safety/compliance forms complete?
- Can the job be invoiced?
- Can the work order be closed?

This creates an expensive administrative tail after every job.

The most valuable employee action should be judgment.

Instead, supervisors spend time checking paperwork.

3. Product Vision

FieldProof should make this possible:

A technician finishes a job, uploads whatever evidence naturally comes from the work, and walks away.

FieldProof should handle everything after that unless it encounters a genuine decision.

A supervisor should not need to inspect every photo, receipt, or checklist.

They should receive something like:

Decision Required

Job FP-1842

Technician reported three filters installed.

Original work order authorized two filters.

Evidence:

- Receipt: 3 × filter
- Installation photos: 3 compatible units detected
- Technician voice note: emergency replacement required
- Additional cost: \$54

Recommended action: Approve additional charge and close job.

Approve

Request Clarification

Reject

Once approved, FieldProof continues autonomously.

4. Target Users

4.1 Primary User — Operations Supervisor

Responsible for reviewing completed work and approving exceptions.

Needs:

- fewer unnecessary reviews,
 - clear evidence,
 - fast decision-making,
 - confidence in automated actions,
 - auditability.
-

4.2 Secondary User — Field Technician

Completes jobs in the field.

Needs:

- minimal paperwork,
 - easy evidence submission,
 - clear requests if something is missing,
 - no repetitive form filling.
-

4.3 Business Administrator

Handles invoices, reports, records, and compliance.

Needs:

- structured closeout records,
 - reliable documentation,
 - fewer incomplete jobs,
 - standardized reports.
-

5. Product Goals

G1 — Autonomous Closeout

Complete the majority of administrative job-closeout steps without human involvement.

G2 — Evidence-Backed Decisions

Every important conclusion must reference the evidence that supports it.

G3 — Human Judgment Only Where Necessary

Escalate only:

- policy conflicts,
- financial exceptions,
- unsupported critical claims,
- ambiguous evidence,
- high-risk actions.

G4 — Proactive Recovery

When information is missing, FieldProof should attempt to obtain it itself before escalating.

G5 — Explainable Automation

Every meaningful action should answer:

- What happened?
- Why did FieldProof do it?
- What evidence supported it?
- Which policy permitted it?
- Did a human approve anything?

G6 — Strong Demoability

A complete workflow should be understandable within a few minutes.

6. Non-Goals

The hackathon MVP will not attempt to:

- replace full ERP systems,
- replace field-service management software,
- perform employee surveillance,
- make irreversible high-value financial decisions autonomously,
- determine legal liability,
- provide safety certifications,
- automatically approve every exception,
- support every industry.

The MVP should prove one workflow extremely well.

7. Core Product Invariant

NO REQUIRED CLAIM MAY ENTER A "VERIFIED" STATE
WITHOUT COMPATIBLE SUPPORTING EVIDENCE
OR EXPLICIT HUMAN RESOLUTION.

And:

NO JOB MAY ENTER "CLOSED"
WHILE AN UNRESOLVED BLOCKING CONFLICT EXISTS.

This invariant is the center of FieldProof.

8. Core Objects

FieldProof operates on five primary concepts.

8.1 Requirement

Something that must be true before the job can close.

Example:

```
{
  "id": "REQ-03",
  "type": "installation_quantity",
  "description": "Install 2 air filters",
  "required": true
}
```

8.2 Claim

Something someone or something says happened.

Example:

```
{
  "id": "CLM-12",
  "type": "part_installed",
  "part": "HVAC-FILTER-A",
  "quantity": 3,
  "source": "technician_voice_note"
}
```

8.3 Evidence

An artifact capable of supporting or contradicting a claim.

Examples:

- image
- video
- receipt
- signature
- PDF
- sensor reading
- voice note
- checklist

8.4 Conflict

A material incompatibility between claims, evidence, requirements, or policy.

Example:

```
Authorized quantity: 2
Reported quantity: 3
Receipt quantity: 3
```

8.5 Decision

A human resolution required before execution can continue.

9. Primary User Journey

Stage 1 — Job Created

A work order contains:

- customer
- location
- technician
- tasks
- expected parts
- spending limit
- evidence requirements
- completion requirements.

Example:

```
Job #1842
```

```
Replace:
```

```
2 × Air Filter A
```

```
Evidence Required:
```

- before photo
- after photo
- customer signature
- parts receipt

```
Authorized Parts:
```

```
2 × Filter A
```


Maximum additional spend without approval:
\$0

10. Technician Completion Flow

Technician clicks:

Complete Job

They upload:

- six images,
- a receipt,
- customer signature,
- voice note.

Example voice note:

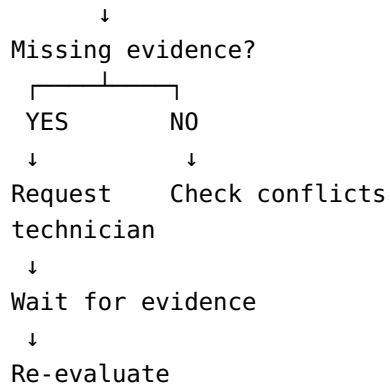
Replaced all three damaged filters. The third one was badly damaged so I replaced it as well.

FieldProof begins automatically.

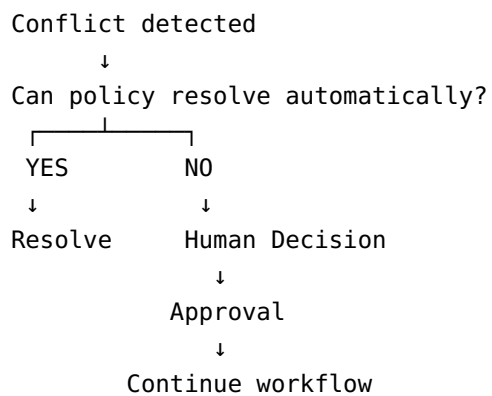
No prompt is required from the supervisor.

11. Autonomous Processing Flow

```
JOB_COMPLETED
  ↓
Load job requirements
  ↓
Collect submitted evidence
  ↓
Extract structured claims
  ↓
Classify evidence
  ↓
Match claims ↔ requirements
  ↓
Validate claim ↔ evidence compatibility
  ↓
Run policy rules
```



For conflicts:



12. Winning Demo Scenario

The demo should intentionally contain both autonomy and escalation.

Work Order

Replace 2 × Filter A.

Maximum authorized quantity: 2.

Required:

- ✓ before photo
- ✓ after photo
- ✓ receipt
- ✓ customer signature

Technician uploads:

- before photo
- receipt for three filters
- customer signature
- voice note
- one incomplete after-photo

FieldProof extracts:

Claim:
3 filters were replaced.

Receipt:
3 filters purchased.

Image evidence:
only two replacement filters confidently visible.

Policy:
only two authorized.

FieldProof first notices that evidence is insufficient.

It automatically sends:

Hi Daniel. I need one additional photo showing the third installed filter before I can complete Job #1842.

Technician uploads the photo.

FieldProof re-runs verification.

It now establishes:

Three filters were installed.

Evidence confidence: HIGH.

But:
Authorized quantity = 2
Actual quantity = 3

FieldProof does not close the job.

Supervisor receives:

DECISION REQUIRED

Job #1842

An additional Filter A was installed outside the original approved scope.

Original authorization:
2 units

Verified installation:
3 units

Additional amount:
\$54

Supporting evidence:
✓ technician note
✓ receipt
✓ installation photos

Approve additional charge?

Supervisor clicks:

Approve

FieldProof automatically:

1. resolves the conflict,
2. records approval,
3. updates billable amount,
4. generates closeout summary,
5. marks evidence requirements satisfied,
6. marks job complete,
7. generates invoice payload,
8. creates final evidence receipt.

This is the complete demo.

13. Functional Requirements

FR-01 — Job Intake

System must accept a structured job/work-order definition.

Must contain:

- job ID,
 - technician,
 - tasks,
 - evidence requirements,
 - authorized quantities,
 - approval rules.
-

FR-02 — Evidence Upload

Users must be able to upload:

- images,
- PDFs,
- receipts,
- audio,
- signatures.

Each artifact receives:

- immutable evidence ID,
 - timestamp,
 - uploader,
 - content hash,
 - evidence type,
 - object-store URL.
-

FR-03 — Evidence Understanding

FieldProof must transform unstructured evidence into structured observations.

Example image output:

```
{  
  "evidence_id": "EVID-201",
```

```
"observations": [  
  {  
    "type": "installed_component",  
    "component": "Filter A",  
    "quantity": 3,  
    "confidence": 0.94  
  }  
]  
}
```

FR-04 — Voice Understanding

Voice notes should be transcribed and transformed into claims.

FR-05 — Receipt Extraction

Receipts should return structured:

- vendor,
 - line items,
 - quantities,
 - prices,
 - total,
 - date.
-

FR-06 — Claim Generation

The system should normalize observations into claims.

FR-07 — Requirement Matching

Each work-order requirement must receive:

```
VERIFIED  
PARTIAL  
UNSUPPORTED  
CONTRADICTED  
NOT_REQUIRED
```

FR-08 — Missing Evidence Recovery

If evidence is missing and can reasonably be supplied by the technician:

FieldProof should request it automatically.

FR-09 — Conflict Detection

Detect at minimum:

- quantity mismatch,
 - price mismatch,
 - missing artifact,
 - unsupported completion claim,
 - task not completed,
 - spending-limit violation.
-

FR-10 — Human Decision Gate

Material exceptions must generate a decision request.

Every request must contain:

- issue,
 - relevant policy,
 - supporting evidence,
 - financial/risk impact,
 - recommended action.
-

FR-11 — Decisions

Available MVP actions:

APPROVE
REJECT
REQUEST_CLARIFICATION

FR-12 — Workflow Resumption

After human approval, the workflow must resume automatically from its paused state.

The supervisor should not need to re-run anything.

FR-13 — Closeout Report

Generate:

- work completed,
 - parts used,
 - evidence summary,
 - exceptions,
 - approvals,
 - final billable amount.
-

FR-14 — Invoice Payload

MVP may simulate external invoicing.

Example:

```
{  
  "job_id": "1842",  
  "customer_id": "CUS-001",  
  "amount": 354,  
  "status": "ready_to_invoice"  
}
```

FR-15 — Evidence Receipt

Every completed job should have a receipt containing:

```
Job  
Actions  
Evidence  
Claims  
Conflicts  
Human decisions
```

Policies
Final outcome
Timestamp

14. Product Screens

Keep the UI extremely focused.

Screen 1 — Operations Dashboard

Cards:

12 Processing
31 Closed Automatically
3 Waiting on Technician
2 Decision Required

Screen 2 — Job Timeline

Example:

10:21 Technician completed job
10:21 FieldProof began verification
10:22 7 evidence artifacts processed
10:22 Missing evidence detected
10:22 Technician contacted automatically
10:27 New image received
10:27 Verification resumed
10:28 Policy conflict detected
10:28 Supervisor approval requested
10:31 Approved by Sarah

10:31 Job closed automatically

This timeline is important.

It visibly proves autonomy.

Screen 3 — Decision Card

The hero UI.

Show:

- issue,
 - recommended action,
 - financial impact,
 - evidence,
 - policy,
 - approve/reject buttons.
-

Screen 4 — Evidence Graph

Visual relationship:

```
graph TD
    Requirement --> Claim
    Claim --> Photo
    Claim --> Receipt
    Photo --> Verified
    Receipt --> Verified
```

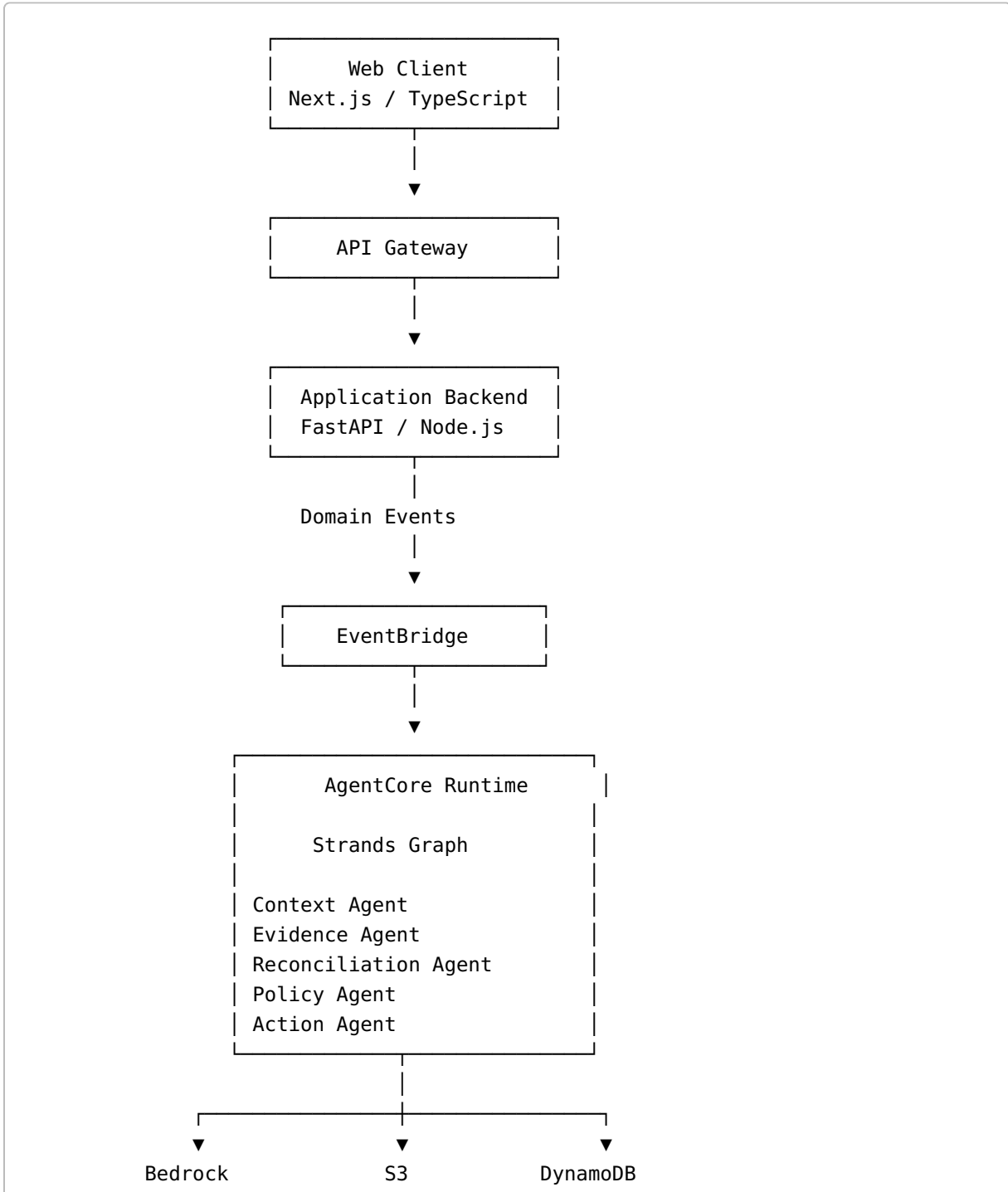
Screen 5 — Receipt

A final proof page showing why the job closed.

15. Technical Requirements Document

15.1 System Architecture

Recommended architecture:



↓
Evidence Objects

15.2 Why Multi-Agent

Agents should have specific responsibilities.

Avoid five agents independently discussing the problem.

Use deterministic orchestration.

16. Agent Roles

Agent 1 — Context Agent

Responsible for understanding the work order.

Input:

```
{  
  "job_id": "1842"  
}
```

Output:

```
{  
  "requirements": [],  
  "policies": [],  
  "allowed_actions": []  
}
```

Agent 2 — Evidence Agent

Responsible for understanding uploaded artifacts.

Tools:

```
read_image
transcribe_audio
extract_receipt
read_document
calculate_hash
```

Output:

```
{
  "claims": [],
  "observations": []
}
```

Agent 3 — Reconciliation Agent

Compares:

```
requirements
vs
claims
vs
evidence
```

Produces:

```
verified requirements
missing requirements
conflicts
confidence
```

Agent 4 — Policy Agent

Determines whether a discrepancy:

```
AUTO_RESOLVE
REQUEST_EVIDENCE
```

REQUIRE_HUMAN
BLOCK

The agent may interpret policy.

The final authorization check must be deterministic.

Agent 5 — Action Agent

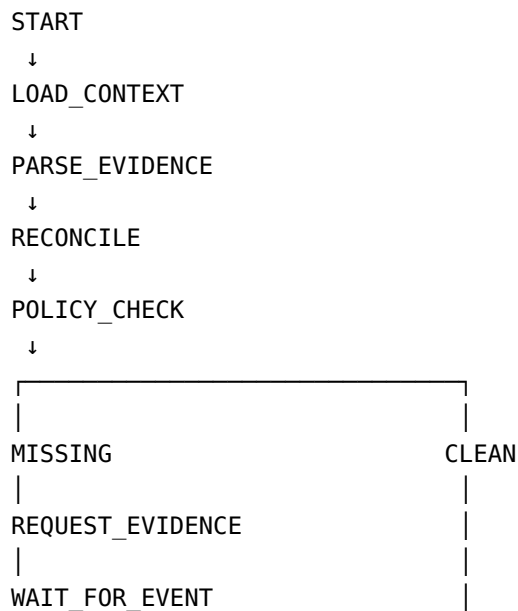
Performs permitted external actions.

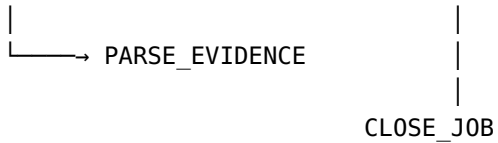
Examples:

```
request_more_evidence()
create_decision()
close_job()
generate_report()
create_invoice()
notify_customer()
```

17. Agent Graph

Recommended graph:





Conflict branch:

```
POLICY_CHECK
  ↓
CONFLICT
  ↓
HUMAN_DECISION
  ↓
PAUSE
  ↓
decision event
  ↓
RESUME
  ↓
ACTION_AGENT
```

18. Deterministic Safety Boundary

The LLM should not have unlimited authority.

Before any action executes:

```
authorize(action, context)
```

must pass.

Conceptually:

```
if action.type == "close_job":
    assert unresolved_blocking_conflicts == 0
    assert all_required_requirements_verified

if action.type == "increase_invoice":
    assert human_approval_exists
```

```
if action.type == "request_evidence":  
    assert requested_evidence_is_required
```

LLM proposes.

Policy layer authorizes.

Tool executes.

19. Event Architecture

Primary event types:

```
JOB_CREATED  
JOB_COMPLETED  
EVIDENCE_UPLOADED  
EVIDENCE_PROCESSED  
MISSING_EVIDENCE_DETECTED  
EVIDENCE_REQUESTED  
CONFLICT_DETECTED  
DECISION_REQUESTED  
DECISION_RESOLVED  
JOB_VERIFIED  
JOB_CLOSED
```

Example:

```
{  
  "event": "EVIDENCE_UPLOADED",  
  "event_id": "EVT-92831",  
  "job_id": "JOB-1842",  
  "evidence_id": "EVID-201",  
  "timestamp": "..."  
}
```

20. Workflow State Machine

Job states:


```
OPEN
IN_PROGRESS
SUBMITTED
VERIFYING
WAITING_FOR_EVIDENCE
WAITING_FOR_DECISION
VERIFIED
CLOSING
CLOSED
FAILED
```

Valid example:

```
IN_PROGRESS
→ SUBMITTED
→ VERIFYING
→ WAITING_FOR_EVIDENCE
→ VERIFYING
→ WAITING_FOR_DECISION
→ VERIFIED
→ CLOSING
→ CLOSED
```

21. Data Model

Jobs

```
id
customer_id
technician_id
status
description
authorized_amount
final_amount
created_at
completed_at
closed_at
```

Requirements

```
id
job_id
type
description
required
status
```

Evidence

```
id
job_id
type
storage_url
sha256
uploaded_by
metadata
created_at
```

Claims

```
id
job_id
evidence_id
type
value
confidence
status
```

ClaimEvidenceLinks

```
claim_id
evidence_id
relationship
confidence
```

Relationships:

```
SUPPORTS
CONTRADICTS
PARTIALLY_SUPPORTS
```

Conflicts

```
id
job_id
type
severity
description
status
policy_id
```

Decisions

```
id
job_id
conflict_id
requested_from
status
decision
comment
resolved_at
```

Events

```
id
job_id
type
payload
created_at
```

22. Suggested DynamoDB Layout

For hackathon speed, one-table design can be used.

Partition:

```
PK = JOB#1842
```

Sort keys:

```
JOB  
REQ#001  
EVIDENCE#001  
CLAIM#001  
CONFLICT#001  
DECISION#001  
EVENT#20260910...
```

This lets the application load the complete job state efficiently.

23. API Specification

Create Job

```
POST /api/jobs
```

Get Job

```
GET /api/jobs/:jobId
```

Complete Job

```
POST /api/jobs/:jobId/complete
```

Upload Evidence

```
POST /api/jobs/:jobId/evidence
```

Get Timeline

```
GET /api/jobs/:jobId/events
```

Get Decision

```
GET /api/decisions/:decisionId
```

Resolve Decision

```
POST /api/decisions/:decisionId
```

Payload:

```
{  
  "decision": "APPROVE"  
}
```

Get Receipt

```
GET /api/jobs/:jobId/receipt
```

24. Tools Exposed to Agents

Never expose raw database access.

Use narrow tools.

Example:

```
get_job_context(job_id)

get_evidence(evidence_id)

save_claim(...)

request_technician_evidence(...)

create_conflict(...)

request_human_decision(...)

generate_closeout_report(...)

prepare_invoice(...)

close_job(...)
```

Each tool should:

1. validate inputs,
2. verify authorization,
3. execute action,
4. emit event,
5. return structured result.

25. Evidence Compatibility

Not every artifact proves every claim.

Example:

Claim	Valid evidence
Part purchased	Receipt
Part installed	Installation photo/video
Customer accepted	Signature
Job site visited	Timestamp/geolocation/photo

Claim	Valid evidence
Price paid	Receipt
Technician statement	Voice/text note

A receipt can prove:

Three filters were purchased.

It does **not** prove:

Three filters were installed.

This distinction makes FieldProof significantly stronger technically.

26. Confidence Rules

Each machine-extracted observation receives a confidence value.

Recommended:

0.90–1.00	HIGH
0.70–0.89	MEDIUM
<0.70	LOW

Critical requirements should not automatically verify from low-confidence evidence.

Example:

```
IF evidence.confidence < 0.70
THEN REQUEST_EVIDENCE or HUMAN_REVIEW
```

27. Conflict Severity

```
INFO
WARNING
BLOCKING
```

Examples:

INFO

Technician finished five minutes later than expected.

WARNING

Receipt date differs slightly from job date.

BLOCKING

Unauthorized additional charge.

Missing customer approval.

Unsupported completion claim.

28. Human-in-the-Loop Protocol

A decision must contain structured context.

```
{
  "decision_id": "DEC-009",
  "job_id": "JOB-1842",
  "question": "Approve one additional Filter A?",
  "recommended_action": "APPROVE",
  "financial_impact": 54,
  "evidence_ids": [
    "EVID-1",
    "EVID-2",
    "EVID-3"
  ],
  "policy": "Parts outside original scope require supervisor approval."
}
```

29. Audit Receipt

Example:


```
{
  "receipt_id": "RCP-1842",
  "job_id": "JOB-1842",
  "result": "CLOSED",
  "requirements": {
    "total": 5,
    "verified": 5
  },
  "conflicts": [
    {
      "id": "CON-1",
      "resolution": "HUMAN_APPROVED",
      "decision_id": "DEC-009"
    }
  ],
  "actions": [
    "REQUESTED_EVIDENCE",
    "GENERATED_REPORT",
    "CREATED_INVOICE",
    "CLOSED_JOB"
  ]
}
```

Optionally hash the canonicalized receipt:

```
SHA256(receipt)
```

This makes tampering evident even without blockchain complexity.

30. AWS Stack

Recommended MVP:

Agent Runtime

Amazon Bedrock AgentCore

Runs the FieldProof agent workflow.

Agent Framework

Strands Agents

Graph-based orchestration.

Models

Amazon Bedrock

Use an appropriate multimodal model for:

- reasoning,
- image analysis,
- extraction.

Evidence

Amazon S3

Stores:

- images,
- audio,
- receipts,
- reports.

State

Amazon DynamoDB

Stores structured job state.

Events

Amazon EventBridge

Drives asynchronous workflow continuation.

Backend

Either:

AWS Lambda

or a small persistent backend if needed.

Frontend

```
Next.js
TypeScript
Tailwind
```

31. Repository Structure

```
fieldproof/
|
├─ apps/
|   └─ web/
|   └─ api/
|
├─ agents/
|   └─ orchestrator/
|   └─ context/
|   └─ evidence/
|   └─ reconciliation/
|   └─ policy/
|   └─ action/
|
├─ tools/
|   └─ jobs/
|   └─ evidence/
|   └─ decisions/
|   └─ reports/
|
├─ domain/
|   └─ claims/
|   └─ policies/
|   └─ reconciliation/
|   └─ authorization/
|
├─ infra/
|
├─ tests/
|
└─ demo/
```

Keep core deterministic logic outside prompts.

32. Critical Invariants to Test

INV-001

A job cannot close with unresolved blocking conflicts.

INV-002

A required requirement cannot be silently skipped.

INV-003

Evidence cannot support a claim if its evidence type is incompatible.

INV-004

Financial exceptions cannot execute without appropriate authorization.

INV-005

Duplicate events must not duplicate actions.

INV-006

A workflow waiting for human approval must resume exactly once.

INV-007

Every executed action must generate an audit event.

INV-008

Every finalized claim must retain its supporting evidence references.

INV-009

Deleting or replacing an artifact must invalidate dependent conclusions.

INV-010

Failed agent reasoning must never bypass deterministic policy rules.

33. Idempotency

Agentic workflows are event-driven.

Duplicate delivery must be expected.

Every side-effecting request should include:

```
idempotency_key
```

For example:

```
close_job:JOB-1842:v1
```

Repeated calls should return the already-created result.

This prevents:

- duplicate invoices,
- duplicate messages,
- duplicate decisions,
- duplicate closeouts.

34. Security

For MVP:

- signed upload URLs,
- scoped job access,
- server-side authorization,
- sanitized tool input,
- secrets stored outside source code,
- evidence access through authenticated endpoints.

Agent prompts should never contain credentials.

35. Observability

Every run should expose:

```
workflow ID
agent
input event
tools called
duration
result
error
```

The frontend timeline should use the same events.

This means observability doubles as product UI.

36. Failure Handling

If extraction fails:

```
retry
→ alternate parsing
→ human fallback
```

If an external action fails:

```
JOB remains VERIFIED
invoice_action = FAILED
```

Do not incorrectly mark the entire job incomplete.

37. Evaluation Dataset

Create approximately 15–25 fixture jobs.

Include:

Happy cases

Everything correct.

Missing evidence

Missing after-photo.

Contradiction

Technician says 3; receipt says 2.

Unauthorized work

Three parts installed; two authorized.

Bad evidence

Blurry photo.

Duplicate submission

Same receipt uploaded twice.

Human rejection

Supervisor denies additional cost.

Resumption

Evidence arrives after workflow pauses.

38. Core Metrics

Show these in the demo.

Example:

Steps in workflow:	19
Handled autonomously:	17
Technician interactions:	1
Supervisor decisions:	1
Manual document reviews:	0

Other useful metrics:

Automation rate
Average human interventions/job
Evidence completeness rate
Autonomous recovery rate
Average closeout time

39. MVP Scope

For the hackathon, only implement:

Required

- job creation
- evidence upload
- multimodal evidence parsing
- claim generation
- requirement verification
- missing evidence request
- event-driven workflow continuation
- one policy conflict
- human approval
- workflow resumption
- automatic closeout
- receipt
- beautiful timeline

Simulated Integrations

Invoice provider.

SMS/email service if necessary.

ERP.

Do not waste time implementing integrations that do not improve the core demonstration.

40. Priority Order

Build in this order:

P0

1. job schema
2. evidence schema
3. workflow state machine
4. agent orchestration
5. evidence extraction
6. reconciliation engine
7. conflict generation
8. decision gate
9. workflow resumption
10. close-job invariant

P1

1. evidence-request messaging
2. timeline UI
3. decision UI
4. final receipt
5. metrics

P2

1. animations
2. extra policies
3. external integrations
4. additional industries

41. What Not to Overbuild

Do not spend the hackathon implementing:

- ten different industries,
- blockchain,
- complex organization management,
- payments,
- real ERP connectors,
- complicated permissions,
- AI chat,
- mobile native apps,
- twenty agent personas.

The win condition is proving:

```
messy real-world evidence
  ↓
autonomous reasoning
  ↓
agent attempts recovery
  ↓
one genuine human decision
  ↓
autonomous completion
  ↓
auditable result
```

42. Demo Script

Scene 1

"This technician has finished an HVAC job."

Show work order.

Two filters authorized.

Scene 2

Technician submits evidence.

No supervisor involvement.

Click complete.

Scene 3

FieldProof begins working.

Timeline starts filling automatically.

```
Loaded job context
Processed receipt
Analyzed 5 images
```

Transcribed voice note
Generated 7 claims

Scene 4

FieldProof catches missing evidence.

Third installed filter is not sufficiently visible.

It contacts technician automatically.

Do not click anything as supervisor.

Scene 5

Simulate technician upload.

Workflow resumes automatically.

New evidence received
Reconciliation resumed
3 installations now verified

Scene 6

Conflict appears.

3 installed
2 authorized
+\$54

Only now does the human become involved.

Scene 7

Click:

Approve

Do not click anything else.

Let the agent continue.

Scene 8

Timeline:

```
Approval recorded
Invoice updated
Closeout report generated
Customer package created
Job closed
Receipt generated
```

Scene 9

Finish with:

```
19 workflow steps
17 autonomous
1 technician evidence request
1 supervisor decision
0 manual document reviews
```

Then:

FieldProof doesn't ask humans to supervise AI.

It lets AI do the administrative work and brings humans back only when human judgment actually matters.

43. Product Positioning

One-liner

FieldProof is the autonomous closeout agent for field-service teams.

Extended

FieldProof reads field evidence, verifies that work matches the job, resolves missing documentation, escalates genuine exceptions, and completes the administrative workflow automatically.

Tagline

Finish the paperwork when the fieldwork finishes.

Alternative:

The agent that finishes the job after the technician leaves.

44. Differentiation

FieldProof is not:

An AI form filler

It reasons across conflicting evidence.

A chatbot

It operates asynchronously from events.

An OCR tool

Understanding documents is only one step.

A workflow automation builder

It can interpret messy real-world evidence and decide what action is required.

A fully autonomous authority

Human judgment remains at material decision boundaries.

The differentiation is:

Evidence-aware autonomous execution with explicit human decision gates.

45. Future Expansion

After the hackathon:

Industries

- HVAC
- telecom
- solar
- utilities
- construction
- property management
- industrial maintenance

Integrations

- ServiceTitan
- Salesforce Field Service
- SAP
- Oracle
- QuickBooks
- Stripe
- Twilio

Capabilities

- warranty verification,
- fraud/anomaly detection,
- SLA compliance,
- automated customer follow-ups,
- technician quality insights,
- recurring failure detection,
- predictive parts requirements.

46. North Star

FieldProof succeeds when the normal experience becomes:

Technician:
"I've finished."

FieldProof:
"I'll handle the rest."

Supervisor:

Only sees something when their judgment is actually needed.

The product should demonstrate not simply that an agent can reason, but that an agent can **own a real workflow, gather what it needs, stop when authority is required, and continue to completion afterward.**